

10. (a) In the State pattern, the current state of the context will change as the program runs, and the behaviour will change for each different state. In the Strategy pattern, the behaviour of the program should not change for each strategy, the result will just be computed using a different algorithm.
- (b) i. If an **Academic** needs to change from a **Lecturer** to a **Professor**, then a new object has to be created, have all the fields of the previous object copied over, and the old object deleted.
- ii. When the rank changes, the current **mRank** object needs to be deleted, but if a reference to it has been given out, then even though it is not in use there may be a dangling reference, and so the object will not be deleted.
- iii. The open-closed principle can allow the creation of immutable objects, as the object being "closed" means it should not be able to be modified, and this can be done by providing no methods that change state in the interface. This could still cause the issue with dangling references.
- (c) i. **draw()** should be an abstract method as every **Shape** subclass should be required to implement it, as it does not make sense to have a shape that cannot be drawn.
- (d) Create two new interfaces, **Wrapped** and **Unwrapped**. Create components using generics, designed to be used with **Wrapped** and **Unwrapped**. When writing a decorator, write it so that it takes in a **Unwrapped** and returns a **Wrapped**.

1 2022 Paper 1 Question 3

- (a) A disadvantage of immutable classes is that changing of data requires deleting an object and creating a new one, which is more expensive than modifying fields. Advantages are reduction of bugs, thread safety by default, and no side effects.
- (b) (i) This is not a good solution as if a user wants to know if a class can be modified, a parameter has to be tested, instead of the name being used to easily identify whether a class can be mutated or not. Ideally, an immutable class should not provide methods that could modify state, while this implementation does provide methods that appear as if they could modify state even if they do not.
- (ii) Instead, an immutable version of the class called **ImmutableAssetLocation** should be provided, to clearly indicate to the user which one is immutable and which one is not. The immutable version would not provide setters, as its state cannot be modified.



<https://www.cl.cam.ac.uk/teaching/exams/pastpapers/y2022p1q3.pdf>

2 2022 Paper 1 Question 4

- (a) Type checking is used to ensure that every value that is used in a method matches said method's signature. For example, if we have **List<A>**, and a **List<String>** is declared, type checking ensures that any value inserted into said list is in fact a **String**.
- (b) Type erasure is the process of compiling a program that uses generics by using type checking to ensure that all types match and no conflicts occur, then deleting all information relating to the type of generic classes declared, that way even if data about types used is declared, the JVM has already ensured that all types match. This reduces the memory footprint of other approaches, such as declaring a different class for every type, and instead only declaring types that are used.
- (c) (i) This is true as **Number** is a superclass of **Integer**.



<https://www.cl.cam.ac.uk/teaching/exams/pastpapers/y2022p1q4.pdf>



- (ii) This is true as `Number` is a superclass of `Integer`.
 - (iii) This is true as `Number` is a superclass of `Integer`
 - (iv) This is not true as any given class is not guaranteed to be a superclass of `Number`, i.e. is not guaranteed to have an is-a relationship to `Number`.
 - (v) This is not true as any given class is not guaranteed to be a superclass of `Number`, i.e. is not guaranteed to have an is-a relationship to `Number`.
- (d)
- (i) This will not compile as `<[type]>` is not used to specify a type.
 - (ii) This will compile, and is more verbose than (iii) as it specifies `Object`, but all classes inherit from `Object`. However, it clearly defines the fact that primitives cannot be used in generics.
 - (iii) This will compile, and simply uses the type defined on the LHS to fill in the type, and so is very clear to read.

3 2025 Paper 1 Question 4

- (a) The Open-Closed principle is that classes should not be modifiable, but easy to add functionality to. It reduces risk of regressions, as any current functionality of a class cannot be affected, so even if new functionality is added to a class, all previous code that uses it will work.
- (b) Abstraction: If it is not known how a method performs certain functionality, then it cannot be modified, instead, only an overridden method, or a method with extra functionality can be provided.
Encapsulation: Data about an object cannot be accessed, so therefore, it is impossible to change its state without using the provided method, the only way to change a class is to add functionality.
Inheritance: If a class cannot be directly modified, the easiest way to change/add functionality is to create a subclass and either add a method or override a currently existing method so it acts differently for this subclass.
Polymorphism: Instead of having a different method for every combination of data types of arguments that makes sense, only one method is provided, and the existing implementations cannot be modified, if this method needs to act on a different set of argument types, the method can be redefined, but with the new method signature.
- (c)
- (i) This code will compile, as `sum` is an integer, and `intValue()` is guaranteed to return an integer as well, so all the type checking conforms.
 - (ii) This code will not compile as `numList` could be a list of any subclass of `Number`, not guaranteed to be an integer, so for example if it is a list of `Doubles`, then an integer cannot be inserted.
 - (iii) This code will compile as `6` is an integer, so by Liskov's Substitution Principle, an integer can be used in place of any superclass of integer.
 - (iv) This code will compile as `Number` is a superclass of `Integer`, so can safely be inserted.
 - (v) This code will not compile as even though `numList` contains `Numbers`, it will not necessarily contain types that can be converted into integers.



<https://www.cl.cam.ac.uk/teaching/exams/pastpapers/y2025p1q4.pdf>

